Data Collection and Preprocessing Phase

Date	24 June 2026
Team ID	SWTID-2026-8296
Project Title	Credit Card Approval Prediction
Maximum Marks	6 Marks

Data Exploration and Preprocessing Template

Data has been sourced from Kaggle. Dataset variables will be statistically analyzed to identify patterns and outliers, with Python employed for preprocessing tasks like normalization and feature engineering. Data cleaning will address missing values and outliers, ensuring quality for subsequent analysis and modeling, and forming a strong foundation for insights and predictions.

Section	**Description**
Data Overview	**application_record.csv** Dimension: 438557 rows x 13 columns Descriptive Statistics: `app.describe()` **credit_record.csv** Descriptive Analysis:

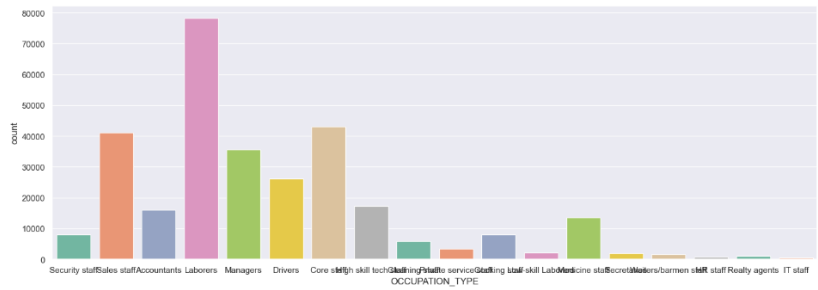
	ID	CNT_CHILDREN	AMT_INCOME_TOTAL	DAYS_BIRTH	DAYS_EMPLOYED	FLAG_MOBIL	FLAG_WORK_PHONE	FLAG_PHONE	FLAG_EMAIL	CNT_FAM_MEMBERS
count	4.385570e+05	438557.000000	4.385570e+05	438557.000000	438557.000000	438557.0	438557.000000	438557.000000	438557.000000	438557.000000
mean	6.022176e+06	0.427390	1.875243e+05	-15997.904649	60563.675328	1.0	0.206133	0.287771	0.108207	2.194465
std	5.716370e+05	0.724882	1.100869e+05	4185.030007	138767.799647	0.0	0.404527	0.452724	0.310642	0.897207
min	5.008804e+06	0.000000	2.610000e+04	-25201.000000	-17531.000000	1.0	0.000000	0.000000	0.000000	1.000000
25%	5.609375e+06	0.000000	1.215000e+05	-19483.000000	-3103.000000	1.0	0.000000	0.000000	0.000000	2.000000
50%	6.047745e+06	0.000000	1.607805e+05	-15630.000000	-1467.000000	1.0	0.000000	0.000000	0.000000	2.000000
75%	6.456971e+06	1.000000	2.250000e+05	-12514.000000	-371.000000	1.0	0.000000	1.000000	0.000000	3.000000
max	7.999952e+06	19.000000	6.750000e+06	-7489.000000	365243.000000	1.0	1.000000	1.000000	1.000000	20.000000

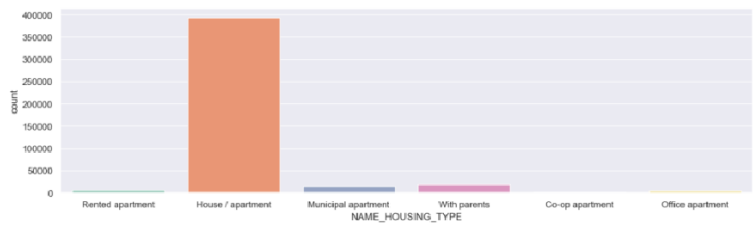
```
credit.describe()
```

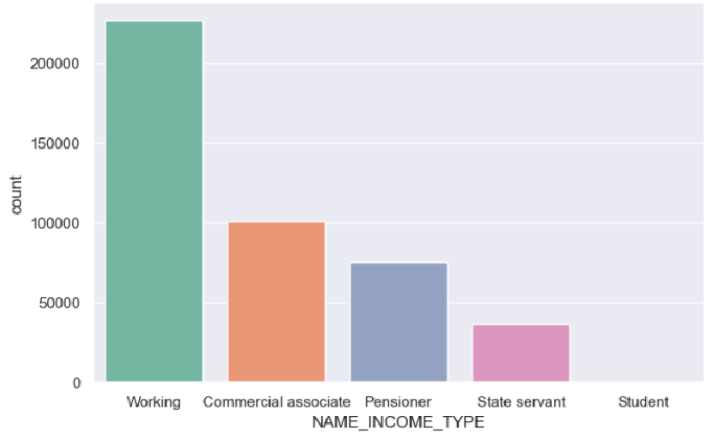
✓ 0.2s

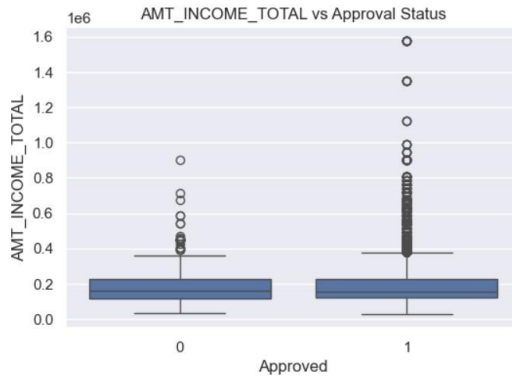
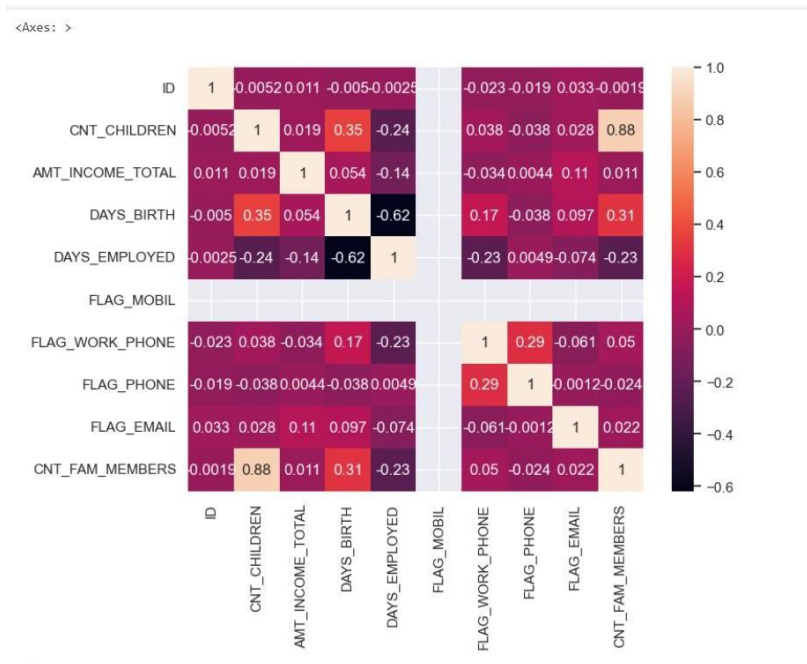
	ID	MONTHS_BALANCE
count	1.048575e+06	1.048575e+06
mean	5.068286e+06	-1.913700e+01
std	4.615058e+04	1.402350e+01
min	5.001711e+06	-6.000000e+01
25%	5.023644e+06	-2.900000e+01
50%	5.062104e+06	-1.700000e+01
75%	5.113856e+06	-7.000000e+00
max	5.150487e+06	0.000000e+00

Univariate Analysis







Bivariate Analysis	 
Multivariate Analysis	
Outliers and Anomalies	Done using feature engineering and data cleaning stages as mentioned in notebook.
Data Preprocessing Code Screenshots	

Loading Data

```
csv_path = "./application_record.csv"  # update with actual file name
app = pd.read_csv(csv_path)
app.head()
```

[7] ✓ 1.1s

ME_TYPE	NAME_EDUCATION_TYPE	NAME_FAMILY_STATUS	NAME_HOUSING_TYPE	DAYS_BIRTH	DAYS_EMPLOYED	FLAG_MOBIL	FLAG_WORK_PHONE	FLAG_PHONE	FLAG_EMAIL
Working	Higher education	Civil marriage	Rented apartment	-12005	-4542	1	1	0	0
Working	Higher education	Civil marriage	Rented apartment	-12005	-4542	1	1	0	0
Working	Secondary / secondary special	Married	House / apartment	-21474	-1134	1	0	0	0
associate	Secondary / secondary special	Single / not married	House / apartment	-19110	-3051	1	0	1	1
associate	Secondary / secondary special	Single / not married	House / apartment	-19110	-3051	1	0	1	1

Handling Missing Data

```
app.isnull().sum()
```

[11] ✓ 0.2s

```
ID                        0
CODE_GENDER               0
FLAG_OWN_CAR              0
FLAG_OWN_REALTY           0
CNT_CHILDREN              0
AMT_INCOME_TOTAL          0
NAME_INCOME_TYPE          0
NAME_EDUCATION_TYPE       0
NAME_FAMILY_STATUS        0
NAME_HOUSING_TYPE         0
DAYS_BIRTH                0
DAYS_EMPLOYED             0
FLAG_MOBIL                0
FLAG_WORK_PHONE           0
FLAG_PHONE                0
FLAG_EMAIL                0
OCCUPATION_TYPE      134203
CNT_FAM_MEMBERS           0
dtype: int64
```

Data Transformation

```
def data_cleansing(data):
    # Adding number of family members with number of children to get overall family members.
    #data['CNT_FAM_MEMBERS'] = data['CNT_FAM_MEMBERS'] + data['CNT_CHILDREN']

    dropped_cols = ['FLAG_MOBIL', 'FLAG_WORK_PHONE', 'FLAG_PHONE',
                    'FLAG_EMAIL', 'OCCUPATION_TYPE', 'CNT_CHILDREN']
    data = data.drop(dropped_cols, axis = 1)

    #converting birth years and days employed to years.
    data['DAYS_BIRTH'] = np.abs(data['DAYS_BIRTH'])/365 #Absolute
    data['DAYS_EMPLOYED'] = data['DAYS_EMPLOYED']/365

    #Cleaning up categorical values to lower the count of dummy variables.
    housing_type = {'House / apartment' : 'House / apartment',
                    'With parents': 'With parents',
                    'Municipal apartment' : 'House / apartment',
                    'Rented apartment' : 'House / apartment',
                    'Office apartment' : 'House / apartment',
                    'Co-op apartment' : 'House / apartment'}

    income_type = {'Commercial associate':'Working',
                   'State servant':'Working',
                   'Working':'Working',
                   'Pensioner':'Pensioner',
                   'Student':'Student'}

    education_type = {'Secondary / secondary special':'secondary',
                      'Lower secondary':'secondary',
                      'Higher education':'Higher education',
                      'Incomplete higher':'Higher education',
                      'Academic degree':'Academic degree'}

    family_status = {'Single / not married':'Single',
                     'Separated':'Single',
                     'Widow':'Single',
                     'Civil marriage':'Married',
                     'Married':'Married'}

    data['NAME_HOUSING_TYPE'] = data['NAME_HOUSING_TYPE'].map(housing_type)
    data['NAME_INCOME_TYPE'] = data['NAME_INCOME_TYPE'].map(income_type)
    data['NAME_EDUCATION_TYPE']=data['NAME_EDUCATION_TYPE'].map(education_type)
    data['NAME_FAMILY_STATUS']=data['NAME_FAMILY_STATUS'].map(family_status)

    return data
data = data_cleansing(app)
data.head()
```

<table>
<tr><td>Feature Engineering</td><td>

Feature Engineering

```
def feature_engineering_target(data):
    good_or_bad = []

    for index, row in data.iterrows():
        paid_off = row['paid_off']
        over_1 = row['pastdue_1-29']
        over_30 = row['pastdue_30-59']
        over_60 = row['pastdue_60-89']
        over_90 = row['pastdue_90-119']
        over_120 = row['pastdue_120-149'] + row['pastdue_over_150']
        no_loan = row['no_loan']

        overall_pastdues = over_1 + over_30 + over_60 + over_90 + over_120

        if overall_pastdues == 0:
            if paid_off >= no_loan or paid_off <= no_loan:
                good_or_bad.append(1)
            elif paid_off == 0 and no_loan == 1:
                good_or_bad.append(1)

        elif overall_pastdues != 0:
            if paid_off > overall_pastdues:
                good_or_bad.append(1)
            elif paid_off <= overall_pastdues:
                good_or_bad.append(0)

        elif paid_off == 0 and no_loan != 0:
            if overall_pastdues <= no_loan or overall_pastdues >= no_loan:
                good_or_bad.append(0)
        else:
            good_or_bad.append(1)

    return good_or_bad
```

```
target = pd.DataFrame()
target['ID'] = pivot_tb.index
target['paid_off'] = pivot_tb['paid_off'].values
target['#_of_pastdues'] = pivot_tb['pastdue_1-29'].values + pivot_tb['pastdue_30-59'].values \
+ pivot_tb['pastdue_60-89'].values + pivot_tb['pastdue_90-119'].values \
+ pivot_tb['pastdue_120-149'].values + pivot_tb['pastdue_over_150'].values

target['no_loan'] = pivot_tb['no_loan'].values
target['target'] = feature_engineering_target(pivot_tb)
credit_app = data_cleansing(app).merge(target, how = 'inner', on = 'ID')
credit_app.drop('ID', axis = 1, inplace = True)
```

</td></tr>
<tr><td>Save Processed Data</td><td>The processed data was saved dynamically in the app i.e. data frame for application_record.csv</td></tr>
</table>